1) Considere o vetor ordenado abaixo em linguagem C:

int v[10] = {2, 5, 8, 12, 17, 23, 28, 35, 41, 50};

0 1 2 3 4 5 6 7 8 9

Qual é o número mínimo de comparações necessário para a busca binária encontrar o elemento 17 nesse vetor?

0.5 pontos

- ⊗ 1
- ⓑ 2
- ⓒ 3
- ⓓ 4
- ⓔ 5

Obs: Meio → índice 4 → 17 (acharia direto).

2) Considere o seguinte código em C:

```c
#include <stdio.h>
#include <stdlib.h>

void altera(int *p) {
    int x = 20;
    *p = *p + x;
    printf("Dentro de altera: *p = %d\n", *p);
}

void processa() {
    int *ptr = (int *) malloc(sizeof(int));
    *ptr = 10;
    altera(ptr);
    printf("Dentro de processa: *ptr = %d\n", *ptr);
    free(ptr);
}

int main() {
    processa();
    return 0;
}
```

Qual a saída esperada para o programa?

0.5 pontos

- ~~a~~ Dentro de altera: *p = 30
  Dentro de processa: *ptr = 30
- b Dentro de altera: *p = 30
  Dentro de processa: *ptr = 10
- c Dentro de altera: *p = 20
  Dentro de processa: *ptr = 30
- d Dentro de altera: *p = 20
  Dentro de processa: *ptr = 10
- e Dentro de altera: *p = 10
  Dentro de processa: *ptr = 20

3) Considere o seguinte código em C:

```c
#include <stdio.h>

void funcC(int x) {
    int c = x + 3;
    printf("C: x = %d, c = %d\n", x, c);
}
void funcB(int y) {
    int b = y * 2;
    funcC(b);
    printf("B: y = %d, b = %d\n", y, b);
}
void funcA(int z) {
    int a = z - 1;
    funcB(a);
    printf("A: z = %d, a = %d\n", z, a);
}
int main() {
    int m = 5;
    funcA(m);
    return 0;
}
```

O desenho da pilha de execução no momento em que a função funcC está em execução é apresentado em:

0.5 pontos

(a) funcC(x = 8, c = 11)
funcB(y = 4, b = 8)
funcA(z = 5, a = 4)
main(m = 5)

(b) main(m = 5)
funcC(x = 8, c = 11)
funcB(y = 4, b = 8)
funcA(z = 5, a = 4)

(c) main(m = 5)
funcA(z = 5, a = 4)
funcC(x = 8, c = 11)

funcC(x = 8, c = 11)
funcB(y = 4, b = 8)

(d) main(m = 5)
funcA(z = 5, a = 4)
funcB(y = 4, b = 8)
funcC(x = 8, c = 11)

4) Considere uma pilha inicialmente vazia. Execute as seguintes operações na ordem indicada:

1. push(10)
2. push(20)
3. push(30)
4. pop()
5. push(40)
6. pop()

I - Qual é o elemento que está no **topo** da pilha após a execução de todas essas operações?

II - Sobre o funcionamento de uma pilha (Stack) está correto?

0.5 pontos

- (a) I - 10 e II - O último elemento inserido é o primeiro a ser removido (LIFO)
- (b) I - 10 e II - O primeiro elemento inserido é o primeiro a ser removido (FIFO)
- (c) I - 10 e II - Todos os elementos podem ser acessados diretamente por índice
- (d) I - 20 e II - A operação de inserção ocorre sempre na base da pilha
- (e) I - 10 e II - O primeiro elemento inserido é o primeiro a ser removido (FIFO)

5) Considere o seguinte código em C:

```c
int main() {
    int a = 50;
    int *ptr = &a;
    int b = *ptr;

    *ptr = 100;

    printf("%d %d", a, b);
    return 0;
}
```

Qual é a saída do programa?

0.5 pontos

- ⓐ 50 50
- ⓑ 100 50
- ⓒ 100 100
- ⓓ 50 100
- ⓔ Erro de compilação

6) Considere os tópicos abaixo:

I - A inserção de um elemento
xx em uma lista sequencial **não ordenada** possui complexidade de pior caso menor do que a inserção de x em uma lista sequencial **ordenada** (Assuma que as listas possuem o mesmo tamanho e podem conter elementos repetidos.)

II - A remoção de um elemento x de uma lista sequencial **não ordenada** possui complexidade de pior caso menor do que a remoção de x de uma lista sequencial **ordenada**. (Assuma que as listas possuem o mesmo tamanho e desconsidere o tempo gasto na busca que localiza a posição de x nas listas.)

É correto afirmar que:

0.5 pontos

- **I - Verdade** — Inserção em lista não ordenada pode ser feita no final (**O(1)**).
  II - Falso — Remoção exige deslocamento em ambos os casos (**O(n)**).
- (b) **I - Falso** — Inserção é sempre **O(n)**.
  II - Verdade — Lista ordenada facilita a remoção.
- (c) **I - Verdade** — Não precisa manter ordem na lista não ordenada.
  II - Verdade — Lista ordenada reduz o custo da remoção.
- (d) **I - Falso** — Inserção tem mesma complexidade nos dois casos.
  II - Falso — Remoção é sempre **O(1)**.
- (e) **I - Verdade** — Inserção direta na lista não ordenada.
  II - Falso — Remoção continua sendo **O(n^2)**.

7) Considere a seguinte afirmação: Para todo inteiro positivo n ≥ 1,

$$\sum_{k=1}^{n} k^2 = \frac{n(n+1)(2n+1)}{6}$$

Prove por princípio da indução matemática:

2.0 pontos

a) $n = 1$

$$\frac{1(1+1)(2\cdot 1+1)}{6} = \frac{1\cdot 2\cdot 3}{6} = 1$$

b) Hipótese

$$\frac{k(k+1)(2k+1)}{6}$$

c) $$\frac{(k+1)(k+2)(2k+3)}{6}$$

$$\frac{k(k+1)(2k+1)}{6} + (k+1)^2$$

$$= k+1)\left(k\left(\frac{2k+1)+6(k+1)}{6}\right)\right.$$

$k(2k+1) = 2k^2+k$

$6(k+1) = 6k+6$

$2k^2+k+6k+6 = 2k^2+7k+6$

$2k^2+7k+6 = (2k+3)(k+2)$

$= (k+1)(k+2)(2k+3)$

6

8) Sobre complexidade de algoritmos e notação Big-O, assinale a alternativa correta:

0.5 pontos

- (a) A complexidade de pior caso representa o menor tempo de execução de um algoritmo.
- (b) A notação Big-O descreve exatamente o tempo de execução de um algoritmo, incluindo constantes.
- (c) A complexidade de pior caso indica o maior número de operações que um algoritmo pode realizar para uma entrada de tamanho nnn, e a notação Big-O representa o crescimento assintótico desse custo.
- (d) A notação Big-O considera todos os termos da função, incluindo constantes e termos de menor ordem.
- (e) A complexidade de pior caso e a notação Big-O são conceitos independentes e não relacionados.

9) Considere os seguintes algoritmos:

- Algoritmo A:
  O(n^2)O(n^2)
- Algoritmo B:
  O(n \log n)O(n log n)
- Algoritmo C:
  O(2^n)O(2^n)
- Algoritmo D: O(n)
- Algoritmo E:
  O(\log n)O(log n)

Assinale a alternativa que apresenta corretamente a ordem crescente de complexidade (do mais eficiente para o menos eficiente), para valores grandes de n.

0,5 pontos

- ~~a~~ E, D, B, A, C
- b D, E, B, A, C
- c E, B, D, A, C
- d E, D, A, B, C
- e C, A, B, D, E

10) Considere a seguinte função em C:

Descreve passo a passo com n = 10

```c
int f(int n) {
    if (n == 0)
        return 0;
    return n + f(n - 1);
}
```

2.0 pontos

$f(0) = 0$

$f(1) = 1 + f(0) = 1$

$f(2) = 2 + f(1) = 3$

$f(3) = 3 + f(2) = 6$

$f(4) = 4 + f(3) = 10$

$f(5) = 5 + f(4) = 15$

$f(6) = 6 + f(5) = 21$

$f(7) = 7 + (f(6)) = 28$

$f(8) = 8 + (f7) = 36$

$f(9) = 9 + f(8) = 45$

$f(10) = 10 + f(9) = 55$